

UNIVERSIDAD TECNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computacion y Diseno Digital

Software (Rediseño) – Aplicaciones Distribuidas [20701] – 7mo Nivel A

Guia de resolución paso a paso

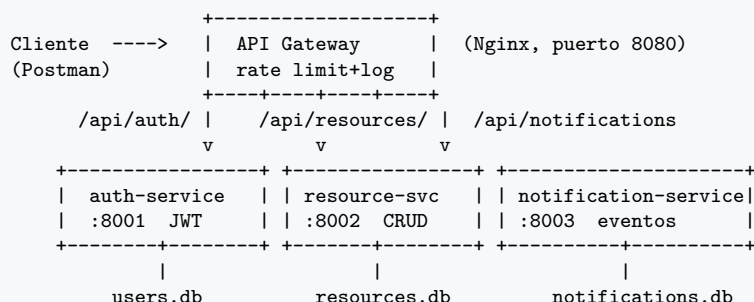
GA-SUM-03 / PE-U2 – Arquitectura, API y Contenerización del Sistema

Objetivo de esta guía. Construir, contenerizar y demostrar un sistema de microservicios REST con autenticación JWT, API Gateway (Nginx) y Docker Compose, que constituye la Entrega 2 del PFC. La solución de referencia usa **FastAPI (Python)**; los mismos pasos aplican a Spring Boot (Java) o Express (Node.js), que también autoriza la guía. Cada equipo debe adaptar el “recurso principal” a su propio PFC (producto, estudiante, laboratorio, ticket, etc.).

Resultado esperado. Al ejecutar `docker compose up --build` deben quedar operativos: tres microservicios con base de datos propia, un API Gateway en el puerto 8080 y un servicio de monitoreo (Portainer) en el 9000, accesibles y verificables con Postman.

1. Arquitectura objetivo

El sistema enruta todas las peticiones del cliente a través del API Gateway, que las distribuye a cada microservicio. Cada microservicio es independiente y posee su propia base de datos (*patrón Database per Service*).



El diagrama C4 nivel 2 (contenedores) y nivel 3 (componentes) se elabora en draw.io o Structurizr y se incluye como figura en el documento LaTeX. El bosquejo anterior solo orienta; no sustituye al C4 formal exigido.

2. Estructura del repositorio

Crea el repositorio en GitHub **público** y respeta esta estructura. La URL debe quedar escrita en el documento de la entrega y el último commit debe corresponder a la fecha y hora de entrega.

```

pfc-e2/
docs/api/          # contratos OpenAPI 3.0 (YAML) exportados de Swagger
  auth.yaml
  resource.yaml
  notification.yaml
auth-service/      # main.py, requirements.txt, Dockerfile
resource-service/
notification-service/
api-gateway/       # nginx.conf
docker-compose.yml
.env.example       # variables SIN valores reales
.gitignore         # debe incluir .env y *.db
informe/           # documento LaTeX de la Entrega 2
README.md
  
```

Paso 0. Preparación del entorno (PowerShell por defecto)

Instala Docker Desktop 4.x, Python 3.11+ y Git. Verifica las versiones antes de empezar.

```
# Windows PowerShell (por defecto en esta guía)
docker --version
docker compose version
python --version
git --version
# Linux/macOS: los mismos comandos funcionan en bash/zsh
```

Paso 1. Diseño de la arquitectura y contratos OpenAPI

Define los tres microservicios y sus responsabilidades: `auth-service` emite y valida JWT; `resource-service` expone el CRUD del recurso principal del PFC y verifica el token en cada operación; `notification-service` registra un evento cada vez que se crea o modifica un recurso.

Para cada microservicio escribe el contrato **OpenAPI 3.0** en [Swagger Editor](#) y exporta el YAML en `docs/api/`. A continuación, un contrato mínimo de `auth-service` como modelo.

```
openapi: 3.0.3
info:
  title: auth-service
  version: "1.0.0"
paths:
  /auth/login:
    post:
      summary: Autentica al usuario y devuelve un JWT
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [username, password]
              properties:
                username: { type: string }
                password: { type: string, format: password }
      responses:
        "200":
          description: Login correcto (JWT en data.access_token)
        "401":
          description: Credenciales invalidas
  /auth/validate:
    get:
      summary: Verifica un token Bearer
      security: [ { bearerAuth: [] } ]
      responses:
        "200": { description: Token valido }
        "401": { description: Token invalido o expirado }
components:
  securitySchemes:
    bearerAuth: { type: http, scheme: bearer, bearerFormat: JWT }
```

Paso 2. Implementación de los microservicios (FastAPI)

Todos los microservicios responden con el sobre JSON `{status, data, message, timestamp}`. El archivo `requirements.txt` (idéntico para los tres, ajustando solo el puerto en el `Dockerfile`) usa versiones verificadas en PyPI.

```
# requirements.txt
fastapi==0.115.0
uvicorn[standard]==0.30.6
sqlalchemy==2.0.34
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
requests==2.32.3
pydantic==2.9.2
```

auth-service/main.py – registro, login (JWT con expiración de 1 h) y validación.

```
import os, datetime as dt
```

```

from fastapi import FastAPI, HTTPException, Header
from pydantic import BaseModel
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import declarative_base, sessionmaker
from passlib.context import CryptContext
from jose import jwt, JWTError

SECRET = os.getenv("JWT_SECRET", "change-me")
ALGO = "HS256"
EXP_MIN = int(os.getenv("JWT_EXP_MIN", "60"))
DB_URL = os.getenv("DATABASE_URL", "sqlite:///./users.db")

engine = create_engine(DB_URL, connect_args={"check_same_thread": False})
Session = sessionmaker(bind=engine)
Base = declarative_base()
pwd = CryptContext(schemes=["bcrypt"], deprecated="auto")

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    username = Column(String, unique=True, index=True)
    password_hash = Column(String)
Base.metadata.create_all(engine)

app = FastAPI(title="auth-service")

def envelope(data=None, message="OK", status="success"):
    return {"status": status, "data": data, "message": message,
            "timestamp": dt.datetime.utcnow().isoformat() + "Z"}

class Credentials(BaseModel):
    username: str
    password: str

@app.post("/auth/register")
def register(c: Credentials):
    db = Session()
    if db.query(User).filter_by(username=c.username).first():
        raise HTTPException(status_code=409, detail="Usuario ya existe")
    db.add(User(username=c.username, password_hash=pwd.hash(c.password)))
    db.commit()
    return envelope({"username": c.username}, "Usuario registrado")

@app.post("/auth/login")
def login(c: Credentials):
    db = Session()
    u = db.query(User).filter_by(username=c.username).first()
    if not u or not pwd.verify(c.password, u.password_hash):
        raise HTTPException(status_code=401, detail="Credenciales invalidas")
    exp = dt.datetime.utcnow() + dt.timedelta(minutes=EXP_MIN)
    token = jwt.encode({"sub": c.username, "exp": exp}, SECRET, algorithm=ALGO)
    return envelope({"access_token": token, "token_type": "bearer",
                    "expires_in": EXP_MIN * 60}, "Login correcto")

@app.get("/auth/validate")
def validate(authorization: str = Header(default="")):
    token = authorization.replace("Bearer ", "")
    try:
        payload = jwt.decode(token, SECRET, algorithms=[ALGO])
    except JWTError:
        raise HTTPException(status_code=401, detail="Token invalido o expirado")
    return envelope({"username": payload["sub"]}, "Token valido")

```

resource-service/main.py – CRUD con verificación de JWT en cada endpoint; al crear o modificar avisa a notification-service.

```

import os, datetime as dt, requests
from fastapi import FastAPI, HTTPException, Header, Depends
from pydantic import BaseModel
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import declarative_base, sessionmaker
from jose import jwt, JWTError

SECRET = os.getenv("JWT_SECRET", "change-me")
ALGO = "HS256"
DB_URL = os.getenv("DATABASE_URL", "sqlite:///./resources.db")

```

```

NOTIF_URL = os.getenv("NOTIFICATION_URL", "http://notification-service:8003")

engine = create_engine(DB_URL, connect_args={"check_same_thread": False})
Session = sessionmaker(bind=engine)
Base = declarative_base()

class Resource(Base):
    __tablename__ = "resources"
    id = Column(Integer, primary_key=True)
    name = Column(String)
    description = Column(String)
Base.metadata.create_all(engine)

app = FastAPI(title="resource-service")

def envelope(data=None, message="OK", status="success"):
    return {"status": status, "data": data, "message": message,
            "timestamp": dt.datetime.utcnow().isoformat() + "Z"}

def require_user(authorization: str = Header(default="")):
    token = authorization.replace("Bearer ", "")
    try:
        return jwt.decode(token, SECRET, algorithms=[ALGO])["sub"]
    except JWTError:
        raise HTTPException(status_code=401, detail="Token requerido o invalido")

class ResourceIn(BaseModel):
    name: str
    description: str = ""

def notify(event, rid, user):
    # Degradacion controlada: si notification-service no responde, no se cae el CRUD
    try:
        requests.post(f"{NOTIF_URL}/notifications",
                      json={"event": event, "resource_id": rid, "user": user},
                      timeout=2)
    except requests.RequestException:
        pass

@app.get("/resources")
def list_resources(user: str = Depends(require_user)):
    db = Session()
    rows = db.query(Resource).all()
    return envelope([{"id": r.id, "name": r.name,
                     "description": r.description} for r in rows])

@app.post("/resources")
def create_resource(body: ResourceIn, user: str = Depends(require_user)):
    db = Session()
    r = Resource(name=body.name, description=body.description)
    db.add(r); db.commit(); db.refresh(r)
    notify("created", r.id, user)
    return envelope({"id": r.id}, "Recurso creado")

@app.put("/resources/{rid}")
def update_resource(rid: int, body: ResourceIn, user: str = Depends(require_user)):
    db = Session(); r = db.query(Resource).get(rid)
    if not r: raise HTTPException(status_code=404, detail="No encontrado")
    r.name, r.description = body.name, body.description; db.commit()
    notify("updated", rid, user)
    return envelope({"id": rid}, "Recurso actualizado")

@app.delete("/resources/{rid}")
def delete_resource(rid: int, user: str = Depends(require_user)):
    db = Session(); r = db.query(Resource).get(rid)
    if not r: raise HTTPException(status_code=404, detail="No encontrado")
    db.delete(r); db.commit()
    return envelope({"id": rid}, "Recurso eliminado")

```

notification-service/main.py – registra cada evento recibido.

```

import os, datetime as dt
from fastapi import FastAPI
from pydantic import BaseModel
from sqlalchemy import create_engine, Column, Integer, String, DateTime
from sqlalchemy.orm import declarative_base, sessionmaker

```

```

DB_URL = os.getenv("DATABASE_URL", "sqlite:///./notifications.db")
engine = create_engine(DB_URL, connect_args={"check_same_thread": False})
Session = sessionmaker(bind=engine)
Base = declarative_base()

class Notification(Base):
    __tablename__ = "notifications"
    id = Column(Integer, primary_key=True)
    event = Column(String); resource_id = Column(Integer)
    user = Column(String); created_at = Column(DateTime, default=dt.datetime.utcnow)
Base.metadata.create_all(engine)

app = FastAPI(title="notification-service")

def envelope(data=None, message="OK", status="success"):
    return {"status": status, "data": data, "message": message,
            "timestamp": dt.datetime.utcnow().isoformat() + "Z"}

class NotificationIn(BaseModel):
    event: str; resource_id: int; user: str

@app.post("/notifications")
def create_notification(n: NotificationIn):
    db = Session()
    row = Notification(event=n.event, resource_id=n.resource_id, user=n.user)
    db.add(row); db.commit(); db.refresh(row)
    return envelope({"id": row.id}, "Notificacion registrada")

@app.get("/notifications")
def list_notifications():
    db = Session(); rows = db.query(Notification).all()
    return envelope([{"id": r.id, "event": r.event,
                     "resource_id": r.resource_id, "user": r.user} for r in rows])

```

Paso 3. API Gateway con Nginx

El Gateway escucha en el puerto 8080, aplica *rate limiting* de 100 peticiones por minuto e IP, registra todas las peticiones y enruta a cada microservicio. Guarda esto como `api-gateway/nginx.conf`.

```

worker_processes auto;
events { worker_connections 1024; }

http {
    # Logging: fecha, metodo, ruta, IP, codigo de respuesta
    log_format apigw '$time_iso8601 $request_method $request_uri $remote_addr $status';
    access_log /var/log/nginx/access.log apigw;

    # Rate limiting: 100 peticiones por minuto por IP
    limit_req_zone $binary_remote_addr zone=apilimit:10m rate=100r/m;

    upstream auth { server auth-service:8001; }
    upstream resources { server resource-service:8002; }
    upstream notifications { server notification-service:8003; }

    server {
        listen 8080;

        location /api/auth/ {
            limit_req zone=apilimit burst=20 nodelay;
            proxy_pass http://auth/auth/; # /api/auth/login -> /auth/login
        }

        # Coleccion (listar/crear): coincidencia exacta
        location = /api/resources {
            limit_req zone=apilimit burst=20 nodelay;
            proxy_pass http://resources/resources;
        }

        # Elemento individual: /api/resources/5 -> /resources/5
        location /api/resources/ {
            limit_req zone=apilimit burst=20 nodelay;
            proxy_pass http://resources/resources/;
        }

        location = /api/notifications {

```

```

        limit_req zone=api limit burst=20 nodelay;
        proxy_pass http://notifications/notifications;
    }
}
}

```

Paso 4. Contenerización con Docker Compose

Cada microservicio usa un **Dockerfile multi-stage** (etapa *builder* para instalar dependencias y etapa *runtime* ligera). El siguiente es el de `auth-service`; para los otros dos solo cambia el `EXPOSE/--port` a 8002 y 8003.

```

# auth-service/Dockerfile
# ===== Stage 1: builder =====
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

# ===== Stage 2: runtime =====
FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY . .
ENV PATH=/root/.local/bin:$PATH
EXPOSE 8001
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8001"]

```

El `docker-compose.yml` levanta los tres microservicios, sus bases de datos (volumenes), el Gateway, el servicio de monitoreo **Portainer** y una red bridge interna.

```

services:
  auth-service:
    build: ./auth-service
    environment:
      - JWT_SECRET=${JWT_SECRET}
      - JWT_EXP_MIN=${JWT_EXP_MIN}
      - DATABASE_URL=sqlite:///data/users.db
    volumes: [auth-data:/data]
    networks: [backend]

  resource-service:
    build: ./resource-service
    environment:
      - JWT_SECRET=${JWT_SECRET}
      - DATABASE_URL=sqlite:///data/resources.db
      - NOTIFICATION_URL=http://notification-service:8003
    depends_on: [notification-service]
    volumes: [resource-data:/data]
    networks: [backend]

  notification-service:
    build: ./notification-service
    environment:
      - DATABASE_URL=sqlite:///data/notifications.db
    volumes: [notif-data:/data]
    networks: [backend]

  api-gateway:
    image: nginx:alpine
    ports: ["8080:8080"]
    volumes:
      - ./api-gateway/nginx.conf:/etc/nginx/nginx.conf:ro
      - gateway-logs:/var/log/nginx
    depends_on: [auth-service, resource-service, notification-service]
    networks: [backend]

  portainer:
    # servicio de monitoreo
    image: portainer/portainer-ce:latest
    ports: ["9000:9000"]
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - portainer-data:/data
    networks: [backend]

```

```
networks:
  backend:
    driver: bridge

volumes:
  auth-data:
  resource-data:
  notif-data:
  gateway-logs:
  portainer-data:
```

Crea `.env.example` (se versiona, sin secretos) y un `.env` local real que **no** se sube (queda en `.gitignore`).

```
# .env.example (committear este; NO el .env real)
JWT_SECRET=pon-aqui-un-secreto-largo-y-aleatorio
JWT_EXP_MIN=60
```

Levanta y verifica el sistema:

```
docker compose up --build          # construye y arranca todo
docker compose ps                  # los 5 servicios deben quedar "running"
# Portainer: abrir http://localhost:9000 para monitorear contenedores
```

Paso 5. Pruebas con Postman (3 flujos end-to-end)

Ejecuta los tres flujos obligatorios contra el Gateway (`http://localhost:8080`). En PowerShell puedes usar `curl.exe`; en Postman crea una coleccion con una variable token.

```
# Flujo 1: registro -> login -> capturar JWT
curl.exe -X POST "http://localhost:8080/api/auth/register" -H "Content-Type: application/json" -d
  "{\"username\":\"demo\",\"password\":\"Demo123*\"}"
curl.exe -X POST "http://localhost:8080/api/auth/login" -H "Content-Type: application/json" -d
  "{\"username\":\"demo\",\"password\":\"Demo123*\"}"
# Copiar data.access_token de la respuesta y guardarlo como $TOKEN

# Flujo 2: sin token (401) -> con token (200) -> POST -> PUT -> DELETE
curl.exe "http://localhost:8080/api/resources" # -> 401
curl.exe "http://localhost:8080/api/resources" -H "Authorization: Bearer $TOKEN" # -> 200
curl.exe -X POST "http://localhost:8080/api/resources" -H "Authorization: Bearer $TOKEN" -H "Content-Type:
  application/json" -d "{\"name\":\"Recurso 1\",\"description\":\"demo\"}"
curl.exe -X PUT "http://localhost:8080/api/resources/1" -H "Authorization: Bearer $TOKEN" -H "Content-Type:
  application/json" -d "{\"name\":\"Recurso 1b\",\"description\":\"editado\"}"
curl.exe -X DELETE "http://localhost:8080/api/resources/1" -H "Authorization: Bearer $TOKEN"

# Flujo 3: crear recurso -> verificar la notificacion generada
curl.exe -X POST "http://localhost:8080/api/resources" -H "Authorization: Bearer $TOKEN" -H "Content-Type:
  application/json" -d "{\"name\":\"Recurso 2\",\"description\":\"con notif\"}"
curl.exe "http://localhost:8080/api/notifications" # debe listar el evento
  "created"
# Linux/macOS: reemplazar curl.exe por curl y usar comillas simples para el -d
```

Documenta los resultados con capturas y una tabla booktabs en el informe. Modelo (rellenar con tus mediciones reales):

Endpoint	Metodo	Cod. esperado	Cod. obtenido	Latencia (ms)
/api/auth/login	POST	200	200	41
/api/resources (s/token)	GET	401	401	8
/api/resources	POST	200	200	23
/api/resources/1	PUT	200	200	19
/api/resources/1	DELETE	200	200	15
/api/notifications	GET	200	200	11

Paso 6. Documento LaTeX de la Entrega 2

El informe (mínimo 12 páginas, compila sin errores) debe incluir: fundamento teórico completo de la Unidad 3 (cuatro subtemas con sus referencias IEEE **verificadas con DOI**, tomadas de la bibliografía de la guía), el diagrama C4 como figura con `caption` y `label`, los contratos OpenAPI en `1stlisting` (YAML), los Dockerfiles comentados, el `nginx.conf`, y la tabla `booktabs` de resultados de pruebas. Incluye además la **declaración de uso de IA** (que herramienta, como y para que) y la **URL del repositorio**.

Paso 7. Repositorio y commit de entrega

Sube todo a GitHub y asegura que el **último commit coincida con la fecha y hora de entrega**; este punto es eliminatorio en la rubrica.

```
git init
git add -A
git commit -m "PFC E2: microservicios + gateway + docker-compose"
git branch -M main
git remote add origin https://github.com/<usuario>/pfc-e2.git
git push -u origin main
git log -1 --format=%cd          # la fecha mostrada debe ser la de entrega
```

Respuestas modelo a las preguntas de análisis (Anexo)

1. ¿Por que cada microservicio debe tener su propia base de datos? El patrón *Database per Service* garantiza el bajo acoplamiento: cada servicio controla su esquema, puede desplegarse y escalar de forma independiente y elige la tecnología de persistencia que mejor se ajuste. Una base compartida reintroduce acoplamiento fuerte (el “monolito distribuido”): un cambio de esquema afecta a varios servicios y se pierde la autonomía. El costo es que las consultas que cruzan servicios requieren composición vía API o eventos, y la consistencia pasa a ser *eventual* en lugar de transaccional.

2. ¿Que ocurre si resource-service cae? Circuit Breaker. Las peticiones quedarían colgadas o devolverían errores en cascada. El patrón *Circuit Breaker* envuelve las llamadas a ese servicio con tres estados: *Closed* (pasa el tráfico), *Open* (tras N fallos consecutivos corta de inmediato y responde con un *fallback* o 503 sin esperar el timeout) y *Half-Open* (deja pasar una prueba para ver si se recupera). En FastAPI puede implementarse con una librería como *pybreaker* alrededor del cliente HTTP, combinada con *timeout* y reintentos acotados. En esta solución, la función `notify()` ya aplica una degradación controlada (timeout de 2 s y *pass* ante fallo) para que la caída de notificaciones no tumbe el CRUD.

3. ¿El API Gateway introduce un cuello de botella? Si: agrega un salto de red (latencia adicional) y es un posible punto único de fallo. Se mitiga escalando Nginx horizontalmente detrás de un balanceador, habilitando *keep-alive* hacia los upstreams, cacheo de respuestas idempotentes y dimensionando `worker_processes` y `worker_connections`. Midiendo con y sin Gateway se comprueba que el sobrecosto suele ser de pocos milisegundos, ampliamente compensado por la centralización de autenticación, *rate limiting*, enrutamiento y observabilidad.

4. ¿Como escalar resource-service a 3 replicas? El servicio debe ser *stateless* (el estado vive en su base de datos). Se escala con Compose y se deja que Nginx balancee por el nombre de servicio (DNS round-robin de Docker):

```
docker compose up --build --scale resource-service=3
```

Requisitos: no fijar `container_name` ni publicar un puerto de host en `resource-service` (para evitar colisiones), y en Nginx mantener el upstream `resources` apuntando al nombre del servicio; Docker resolverá las 3 instancias. Si se requiere control fino, se usa `deploy.replicas` con Swarm o se migra a Kubernetes (Deployment con 3 *Pods* y un *Service*).

Lista de verificación final (mapeada a la rubrica)

Dim.	Verifica que...	Hecho
D1	Diagrama C4 nivel 2 y 3; contratos OpenAPI 3.0 (YAML) en docs/api/	☐
D2	Los 3 microservicios responden; JWT emitido y verificado (Bearer); Gateway con rate limit y logging	☐
D3	Dockerfile multi-stage; docker compose up sin errores; Portainer operativo; .env.example sin secretos	☐
D4	Los 3 flujos Postman documentados con capturas y tabla de latencias	☐
D5	Fundamento teórico U3 (4 subtemas, refs IEEE con DOI) y 4 preguntas respondidas	☐
D6	Informe LaTeX ≥ 12 pag.; URL de GitHub en el documento; último commit = entrega; declaración de IA	☐

Recordatorio de integridad. Las referencias deben ser reales y verificables (DOI/ISBN que corresponda al título); las citas inventadas se penalizan drásticamente. Declara con honestidad el uso de IA: que herramienta, como y para que la usaste.